enabling superpaging does reduce TLB misses for some workloads, it alone is insufficient. For example, `Mcf` still has an L2 TLB MPMI of 57K with THS on, while `Mummer` is unchanged.

5.2. Simulation-Based CoLT Evaluations

5.2.1. Simulated system. Past work on TLBs [5, 6, 9, 27] focuses on miss rates rather than performance because it is infeasible to run memory-intensive applications for long enough durations to provide performance numbers. We also study miss rates, but consider performance too. We use a two-step evaluation to quantify changes in hit rate and to then offer performance numbers feasible for simulations.

Like the bulk of recent work on TLB analysis, we first use a trace-based approach to analyze miss rates [5, 6, 9, 10]. We extract detailed memory traces by simulating an x86 processor on Simics [30]. These highly detailed traces maintain logs of both data and instruction references at the micro-op level. Our traces also capture full-system effects by running benchmarks on a Linux 2.6.38 kernel. We hack the simulated kernel to provide full page table walk details for every single memory reference (this includes the virtual page, the physical page, and all attribute bits). We set the kernel to its default configuration of using THS and normal memory compaction. As we will show, since contiguity is present across all kernel configurations, CoLT will be effective across the range of superpaging and memory compaction settings.

We run the traces through a highly-detailed custom memory simulator. We need to stress our TLBs using simulated workloads in a manner that matches real-system stress; therefore, we use 32-entry and 128-entry L1 and L2 4-way set-associative TLBs. These sizes are chosen as they produce simulated load within 10% of the load experienced by a real system. Our baseline system also assumes a 16-entry fully-associative superpage TLB. As previously detailed, CoLT-FA and CoLT-All reduce this size to 8 entries in order to provide conservative performance improvement data and negate the impact of slightly more complex lookups. Furthermore, unlike past work [9, 11], we model a more realistic TLB hierarchy with 22-entry MMU caches, accessed on TLB misses to accelerate page table walks [5]. Finally, we assume a three-level cache hierarchy similar to the Intel Core i7 (32KB L1 cache, 256KB L2 cache, 4MB LLC).

Having assessed miss rates, we now go beyond prior work and the study the performance implications of our approach. We use the Pin-based [20] CMP$im [18] simulation framework to model a 4-way out-of-order processor with a 128-entry reorder buffer. The processor's TLB and cache parameters match those of our custom trace module. Unfortunately, the simulation speeds of this detailed microarchitectural framework are slow; hence we cannot use it to run full Linux distributions with the memory allocation behavior necessary to study CoLT on sufficiently long-running, large-data applications. However, while this simulator does not maintain virtual-to-physical address translations, it does observe the performance effects of TLB misses by tracking the allocated virtual pages. In tandem with the miss rate eliminations extracted from our trace-based approach, this allows us to interpolate CoLT's actual performance gains. This interpolation strategy is valid for two reasons. First, TLB miss penalties (page walks) are serialized as only one page walk can typically be handled at a time [9, 11]. Hence, TLB misses lie on the execution's critical path. Second, our interpolation approach is actually conservative as it does not account for the instruction replays that would likely occur on TLB misses. Therefore, our projected performance benefits would likely *increase* on a real system.

5.2.2. Evaluation workloads. We use the workloads from Table 1. However, due to slow simulation speeds, we use Simpoints [24] that total to one billion instructions per workload. These simpoints include operating system effects captured by Simics and assume realistic inputs (for Spec, this corresponds to *Ref*).

6. Characterizations of Page Allocation Contiguity

We now quantify how the buddy allocator, memory compaction, THS, and system load affect application contiguity on a real system. We show that page allocation contiguity *always* exists regardless of the kernel configuration.

We begin by discussing the cumulative density functions (CDFs) from Figures 7 to 15. These graph the distribution of contiguities experienced by non-superpage pages. Note that contiguity (the x-axis) is presented as a log scale.

6.1. Superpaging, Memory Compaction

Figures 7, 8, and 9, ordering the benchmarks from highest to lowest TLB MPMI, show contiguity assuming default Linux kernel settings (superpaging and normal memory compaction). The legend provides average contiguity numbers.

Figures 7, 8, and 9 show that there is heavy contiguity across the workloads that cannot be exploited by superpages. On average, pages are in 41-contiguity groupings. Furthermore, there can be large instances of contiguity above the average. For example, most CDFs see many 64 to 256-contiguity instances.

Interestingly, there exist many cases of 512 and 1024-page contiguity. Since THS is enabled, one might initially expect that these should be treated as superpages. However, this contiguity does not translate to superpages for two reasons. First, these memory chunks are not superpage-aligned. Second, THS currently supports superpaging for only *anonymous* pages created through `malloc` calls; as such, a number of file-backed pages created from are not superpage candidates. Overall, we find that 15% of non-superpage pages actually have over 512-page contiguity.

Fortunately, Figures 7, 8, and 9 enjoy particularly high contiguity for TLB-stressing benchmarks. `Mcf`, `Tigr`, and `CactusADM` see tens to hundreds of contiguous pages, indicating their amenability to TLB coalescing. For a number of these benchmarks, such as `Mcf`, high contiguity arises because `malloc` and `mmap` calls are made at the beginning of the execution to allocate large hash-based data structures. These structures span megabytes of space, which the buddy allocator ensures maps to contiguous physical pages.

6.2. No Superpaging, Memory Compaction

Figures 10 to 12 show how contiguity changes when superpaging support is disabled. Average contiguity drops compared to THS on from 41 to 18, for two reasons. First, THS optimistically creates as many 2MB page as possible. While these 2MB pages eventually get broken into 4KB pages due to system load, they do leave large amounts of smaller, residual contiguity. Without THS, contiguity is not generated this way. Second, disabling THS drastically reduces memory compaction daemon invocations. Nevertheless, sufficient exploitable intermediate contiguity remains (in the tens of pages, around 18). Furthermore, heavy TLB-pressure benchmarks like `Mcf` and `Mummer` see very high contiguity.

Surprisingly, some benchmarks like `Omnetpp` and `Sjeng` actually see *higher* contiguity without THS. This occurs because the lack of THS reduces superpages allocated to *other* running processes. As a result, the pages allocated to our workloads remain unfragmented and contiguous.